

Verificación Funcional de Circuitos Integrados

Tecnológico de Costa Rica

Proyecto II - Documentación

Hernández Zárate Amanda

Pérez Ramírez Gabriel

1 Funcionamiento

En esta sección se resume el comportamiento general del dispositivo bajo prueba (DUT). El diseño implementa una red en malla bidimensional de routers, organizada en una matriz. Cada nodo de la malla se conecta con sus nodos adyacentes y, en el contorno, con terminales externos que representan los puntos de acceso al sistema.

La unidad básica de información es un paquete de 40 bits. De forma resumida, el encabezado del paquete contiene los campos de coordenadas de destino (`target_row`, `target_col`), un bit de modo (`mode`) que define si el recorrido se realiza primero por filas (*Row-First*) o primero por columnas (*Column-First*), y un campo de control (`nxt_jump`) que, entre otras cosas, permite distinguir entre envíos unicast y una palabra reservada para *broadcast*. El resto de bits corresponde al *payload*, es decir, a los datos que el usuario desea transportar a través de la malla.

Cada puerto externo está conectado al DUT mediante una interfaz dedicada (`router_external_if`). En dicha interfaz se implementa un protocolo tipo “válido-listo” dividido en dos pares de señales:

- En la dirección de entrada hacia el DUT se emplean las señales `data_out_i_in` y `pndng_i_in`. Cuando el entorno de verificación tiene un paquete listo para ser inyectado, coloca el dato en `data_out_i_in` y levanta `pndng_i_in`. El DUT responde indicando que acepta el paquete mediante la señal `popin`.
- En la dirección de salida desde el DUT se utilizan `data_out` y `pndng`. El DUT levanta `pndng` para indicar que hay un paquete disponible en `data_out`. El entorno confirma que consumió el dato poniendo en alto `pop`.

El enrutamiento entre routers se realiza de manera determinista a partir de las coordenadas de destino y del modo configurado. Cada router compara su identificador de fila y columna con los campos del encabezado, selecciona el siguiente puerto de salida y, en caso de *broadcast*, replica el paquete hacia las direcciones habilitadas. Internamente, cada router cuenta con FIFOs de entrada y salida para desacoplar el tráfico y permitir pequeñas ráfagas sin pérdida de datos.

1.1 Features

A continuación se muestran algunas de las capacidades que posee el dispositivo bajo prueba.

- Enviar paquetes a una dirección específica, ya sea en el modo *Row-First* o *Column-First*.
- Comunicación entre routers adyacentes.
- Comunicación entre routers no adyacentes, esto a través de pasar paquetes por medio de routers que sí son adyacentes hasta que llegue al destino.
- Realización de broadcast.
- Llenado y vaciado de las FIFOs de cada router.

2 TestPlan

Caso de prueba general

Se buscan aleatorizar los siguientes parámetros:

- Terminal origen.
- Terminal destino.
- Mode.

- Target: Fila y columna.
- Payload.
- Número de transacciones por terminal.
- Tiempos de envío de mensajes.

Casos de uso común

- Paquete sencillo punto a punto: Un solo paquete desde un terminal cualquiera a un destino cualquiera. Con esto se verifica que llega al terminal correcto y con payload intacto.
- Tráfico ligero aleatorio: Cada terminal envía pocos paquetes, destinos aleatorios válidos, mode aleatorio.

Casos de esquina

- Máxima ocupación de FIFOs: Generar tráfico continuo hasta que `full_in` / `full_out` se activen.
- Secuencias largas por la misma ruta: Enviar una “rafaga” de paquetes por la misma ruta (mismo origen y destino) con payloads consecutivos.
- Todos hacia el mismo nodo: Muchos paquetes de distintos `src_id` con el mismo `(target_row,target_col)`.
- Enviar un paquete a una dirección inválida, es decir, fuera de la malla.

3 Estructura del ambiente

El ambiente de verificación se construyó siguiendo la metodología UVM, separando la lógica del DUT de los componentes encargados de generar estímulos, observar respuestas y verificar el comportamiento del sistema.

3.1 Módulo estructural `tb` y DUT

El nivel más alto del banco de pruebas está formado por el módulo estructural `tb` y el DUT:

- **Módulo `tb`:** genera la señal de reloj, controla el *reset* síncrono y declara los arreglos de señales que conectan cada terminal externo con el DUT. Además, instancia un arreglo de interfaces `router_external_if` (una por terminal) y las registra en la `uvm.config_db` para que puedan ser tomadas por los distintos agentes.
- **DUT (`mesh_gnrtr`):** implementa la malla de routers parametrizable (número de filas y columnas, ancho del paquete, profundidad de las FIFOs). Expone un conjunto de puertos externos que se conectan directamente con las interfaces `router_external_if`.

3.2 Environment UVM (`mesh_env`)

La clase `mesh_env` agrupa los componentes UVM principales:

- Instancia un arreglo de agentes externos `external_agent`, uno por cada terminal del contorno.
- Instancia un *scoreboard* centralizado encargado de la comparación y las métricas.
- En la fase de conexión enlaza los *analysis ports* de *drivers* y *monitors* con las entradas del *scoreboard*, de forma que todas las transacciones aceptadas u observadas lleguen a un punto común de verificación.

3.3 Agente externo (`external_agent`)

Cada terminal del contorno se modela mediante un agente UVM:

- Agrupa un *sequencer*, un *driver* y un *monitor* que trabajan con objetos de tipo `mesh_pkt`.
- Almacena un identificador de dispositivo `device_id` que indica con qué terminal físico está asociado.
- Durante la fase de construcción obtiene, desde la `uvm.config_db`, el *virtual interface* de tipo `router_external_if` que utilizarán el *driver* y el *monitor*.

3.4 Driver (`mesh_driver`)

El *driver* es el responsable de traducir las transacciones abstractas al protocolo de señales del DUT:

- Recibe objetos `mesh_pkt` desde el *sequencer* y los convierte en un vector de 40 bits sobre la señal `data_out_i_in`, junto con las señales de control asociadas.
- Controla los ciclos de espera antes de inyectar cada paquete mediante el campo `idle_cycles` de la transacción.
- Implementa el *handshake* de entrada usando las señales `pndng_i_in` y `popin`. Solo cuando el DUT confirma la aceptación del paquete se considera completada la transacción.
- Reporta al *scoreboard*, a través de un *analysis port*, cada paquete que el DUT ha aceptado, junto con la información necesaria para el cálculo de latencias.

3.5 Monitor (`monitor`)

El *monitor* observa el comportamiento del DUT en cada terminal:

- Vigila las señales `data_out`, `pndng` y `pop` de la interfaz `router_external_if`.
- Detecta la entrega de un paquete válido cuando `pndng` y `pop` están activos simultáneamente.
- Reconstruye un objeto `mesh_pkt` a partir del vector de 40 bits (`raw_pkt`), separando encabezado y *payload*, y rellena el campo `egress_id` con el terminal por el que se observó la salida.
- Envía el objeto reconstruido al *scoreboard* mediante su *analysis port* para que sea contrastado con los paquetes inyectados.

3.6 Scoreboard

El *scoreboard* compara lo inyectado con lo observado y genera métricas de funcionamiento:

- Recibe, desde los *drivers*, todos los paquetes que el DUT ha aceptado correctamente por cada terminal de entrada.
- Recibe, desde los *monitors*, todos los paquetes que han salido por los terminales externos, junto con su `egress_id`.
- Construye una clave a partir del encabezado del paquete y mantiene colas de paquetes esperados por clave.
- Cuando aparece una observación compatible, calcula la latencia (diferencia en ciclos entre inyección y recepción), actualiza contadores por terminal y marca el resultado de la comparación como válido o fallido.
- Al finalizar la simulación genera un archivo `scoreboard_report.csv` con la información por paquete (origen, destino, terminal de salida, latencia y estado) y un resumen con paquetes esperados, recibidos, perdidos y latencias promedio por terminal.

3.7 Test

La clase `test` orquesta la ejecución de las pruebas en el ambiente:

- Define distintos escenarios de prueba especificando cuántos paquetes envía cada terminal y qué tipo de destinos se desea generar (válidos, inválidos o una mezcla).
- Configura el *scoreboard* con el número total de paquetes que se espera observar durante la simulación.
- Lanza, para cada agente, una secuencia `gen_mesh_seq` que produce objetos `mesh_pkt` de manera aleatoria respetando las restricciones definidas en el TestPlan.
- Monitorea periódicamente el progreso (proporción de paquetes recibidos frente al total esperado) y, si durante un tiempo determinado no hay avance o se supera un tiempo máximo de simulación, fuerza la finalización de la prueba y reporta una posible condición de bloqueo.

3.8 Interfaz

En este proyecto se utiliza la interfaz `router_external_if`, que agrupa en un mismo bloque las señales de entrada y salida:

- **Señales de entrada hacia el DUT:**
 - `data_out_i_in`: paquete de 40 bits a inyectar.
 - `pndng_i_in`: indica que el paquete en `data_out_i_in` es válido y está listo para ser aceptado.
 - `popin`: señal de respuesta del DUT que confirma la aceptación del paquete.
- **Señales de salida desde el DUT:**
 - `data_out`: paquete de 40 bits que el DUT entrega hacia el terminal.
 - `pndng`: indica que hay un dato válido disponible en `data_out`.
 - `pop`: señal generada por el ambiente para confirmar la lectura del paquete.

Dentro de la interfaz se incluyen aserciones que ayudan a validar el uso correcto del protocolo: se comprueba que no se activen señales de *handshake* durante el *reset*, que el DUT solo consuma paquetes cuando `pndng_i_in` está activo y que el dato de salida se mantenga estable mientras `pndng` esté en alto y no se haya solicitado el consumo mediante `pop`.

3.9 Paquetes de transacción

La comunicación entre los distintos módulos del ambiente UVM se modela mediante una única clase de transacción, `mesh_pkt`. Estos objetos viajan:

- Desde el *sequencer* hacia el *driver*, para describir los paquetes que se desean inyectar.
- Desde el *driver* hacia el *scoreboard*, reportando los paquetes que el DUT ha aceptado.
- Desde el *monitor* hacia el *scoreboard*, reportando los paquetes observados a la salida.

Los campos más relevantes de `mesh_pkt` son:

- `nxt_jump`: campo de 8 bits utilizado para el control del paquete, incluyendo un valor reservado para operaciones de *broadcast*.
- `target_row` y `target_col`: campos de 4 bits que especifican la fila y columna de destino dentro de la malla.
- `mode`: bit que define la política de encaminamiento (*Row-First* o *Column-First*).
- `payload`: datos útiles que se desean transportar entre terminales.
- `idle_cycles`: número de ciclos que el *driver* esperará antes de inyectar el paquete; solo es relevante para el ambiente.
- `dest_valid`: indicador de si las coordenadas de destino corresponden a un terminal válido del contorno, útil para clasificar las transacciones durante las pruebas.
- `egress_id`: identificador del terminal por el que se observó la salida del paquete, asignado por el *monitor*.
- `raw_pkt`: representación empaquetada en 40 bits que se corresponde directamente con el vector aplicado o leído en las señales de la interfaz.

3.10 Diagrama del ambiente

En la Fig. 3 se observan los distintos transactores y las conexiones entre estos, para formar el ambiente de pruebas.

4 Comandos de compilación

Para correr el proyecto en el servidor, luego de cargar las herramientas de Synopsys, se debe de usar el código mostrado en 1 para realizar la compilación. Mientras que para correr el archivo ejecutable, se usa lo que se observa en 2. Finalmente, para abrir el debugger, en donde se puede observar los porcentajes de cobertura, con el comando que se observa en 3.

Listing 1: Alias para compilar el testbench en VCS

```
vcs -Mupdate tb.sv -o salida -full64 -sverilog -ntb_opts uvm +incdir+$UVM_HOME/src $UVM_HOME/  
↪ src/uvvm_pkg.sv -kdb -lca -debug_acc+all -debug_region+cell+encrypt -l log_test +lint=  
↪ TFIPC-L -cm line+tgl+cond+fsm+branch+assert
```

Listing 2: Alias para compilar el testbench en VCS

```
./salida -cm line+tgl+cond+fsm+branch+assert
```

Listing 3: Alias para abrir el debugger

```
verdi -cov -covdir salida.vdb
```

5 Resultados obtenidos

La prueba principal que se realizó fue la de enviar 3105 paquetes a direcciones válidas, es decir, a otros agentes. De esos, el scoreboard no procesó 16 por parte de los monitores. El resumen de logs que genera UVM se muestra en 4. Mientras que en 5 se muestran los tiempos de retardo promedios que tiene cada terminal. La cobertura obtenida en esta prueba se observa en la Fig. 2.

Mencionar que se usa el código *graficar.py* para generar la gráfica que muestra los tiempos de retardo en las distintas terminales, Fig. 1. Este código usa el .csv que genera el ambiente UVM.

Listing 4: Resumen de resultados generados por UVM.

```
** Report counts by severity  
UVM_INFO : 21823  
UVM_WARNING : 1  
UVM_ERROR : 35  
UVM_FATAL : 0  
** Report counts by id  
[DRV] 3105  
[ENV] 18  
[LAT] 3089  
[LAT_SUMMARY] 17  
[MON] 3121  
[RNTS] 1  
[SCB_BUFFER_PENDING] 1  
[SCB_CSV] 4  
[SCB_IN] 3105  
[SCB_LOST] 16  
[SCB_OK] 3089  
[SCB_PENDING] 16  
[SCB_REPORT] 6  
[SCB_SYNC] 3093  
[SEQ] 3105  
[TEST] 56  
[TEST_DONE] 1  
[TEST_MONITOR] 1  
[TEST_PROGRESS] 10  
[TEST_SETUP] 1  
[TEST_SYNC] 2  
[TEST_TIMEOUT] 1  
[UVMTOP] 1
```

Listing 5: Resumen de tiempos de retardo de prueba 1.

```
UVM_INFO scoreboard.sv(486) @ 589735: uvvm_test_top.env.scb [LAT_SUMMARY] Termination  
I 0 -> Avg latency = 261 ns (samples=198)
```

```

UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 1 -> Avg latency = 246 ns (samples=168)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 2 -> Avg latency = 225 ns (samples=200)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 3 -> Avg latency = 276 ns (samples=187)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 4 -> Avg latency = 277 ns (samples=201)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 5 -> Avg latency = 235 ns (samples=179)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 6 -> Avg latency = 233 ns (samples=205)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 7 -> Avg latency = 263 ns (samples=188)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 8 -> Avg latency = 260 ns (samples=197)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 9 -> Avg latency = 228 ns (samples=193)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 10 -> Avg latency = 247 ns (samples=210)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 11 -> Avg latency = 266 ns (samples=186)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 12 -> Avg latency = 278 ns (samples=203)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 13 -> Avg latency = 229 ns (samples=183)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 14 -> Avg latency = 222 ns (samples=193)
UVM_INFO scoreboard.sv(486) @ 589735: uvm_test_top.env.scb [LAT_SUMMARY] Terminacion
I 15 -> Avg latency = 255 ns (samples=198)

```

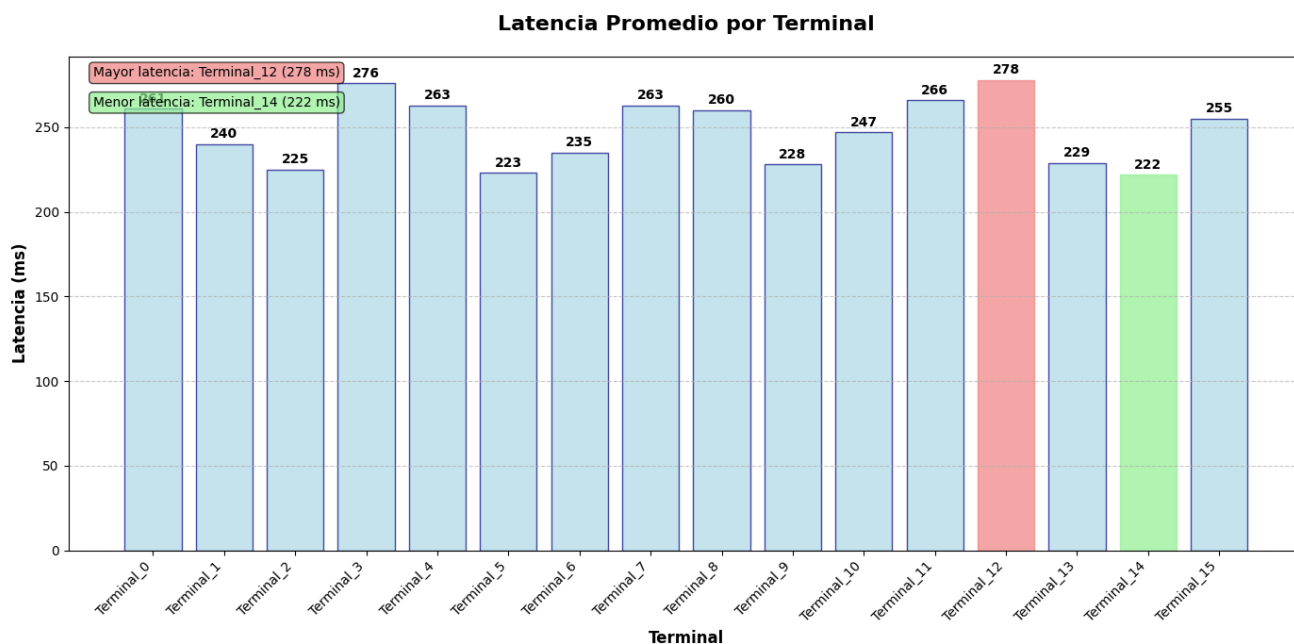


Fig. 1: Tiempos de retardo por terminal.



Fig. 2: Cobertura obtenida en la prueba 1.

Una prueba que se realizó fue la de realizar dos de las pruebas con solo datos que vayan a direcciones válidas, es decir, que vayan a agentes. Y la última prueba fue con datos inválidos, es decir, se quedan atrapados en la malla de routers. Esto se puede cambiar en el código que se muestra en 6. Este código se encuentra en el archivo *Test.sv*, y la línea *seq.dest_mode = 0*; es la que define el tipo de destino que se tiene. Siendo el 0 datos válidos, el

1 datos inválidos y el 2 un compromiso entre ambos.

Listing 6: Código dentro del test, que define el tipo de dirección objetivo de los paquetes.

```
virtual task run_single_test(test_config_t configuration);
fork
  for (int agent_id = 0; agent_id < 'NUM_DEVS; agent_id++) begin
    automatic int agent = agent_id;
    if (configuration.num_packets_per_agent[agent] > 0) begin
      begin
        gen_mesh_seq seq;
        seq = gen_mesh_seq::type_id::create($sformatf("seq_%0d_%s",
                                                    agent,
                                                    configuration.name));
        seq.num = configuration.num_packets_per_agent[agent];

        if (configuration.name == "Prueba_1") begin
          seq.dest_mode = 0;
        end
        else if (configuration.name == "Prueba_2") begin
          seq.dest_mode = 0;
        end
        else if (configuration.name == "Prueba_3") begin
          seq.dest_mode = 1;
        end

        seq.start(env.agents[agent].s0);

        'uvm_info("TEST",
          $sformatf("Agente_%0d_completado:%0d_paquetes_(dest_mode=%0d)",
                    agent, seq.num, seq.dest_mode),
          UVM_MEDIUM)
      end
    end
    else begin
      'uvm_info("TEST", $sformatf("Agente_%0d:_0_paquetes_omitido", agent), UVM_HIGH)
    end
  end
join

#100;
endtask
```

Al realizar esta prueba, se dice que hay 474 que no llegan a ningún monitor, es decir, se quedaron atrapados dentro de la malla de routers. Esto se puede ver en 7.

Listing 7: Resultados de prueba con direcciones objetivo inválidos.

```
** Report counts by id
[DRV] 3105
[ENV] 18
[LAT] 2631
[LAT_SUMMARY] 17
[MON] 2661
[RNTS] 1
[SCB_BUFFER_PENDING] 1
[SCB_CSV] 4
[SCB_IN] 3105
[SCB_LOST] 474
[SCB_OK] 2631
[SCB_PENDING] 474
[SCB_REPORT] 6
[SCB_SYNC] 2635
[SEQ] 3105
[TEST] 56
```

6 Limitaciones

No se exploraron de forma exhaustiva escenarios de tráfico muy prolongados ni configuraciones extremas de congestión.

El *scoreboard* se enfocó en comparar encabezados, *payloads* y latencias básicas, pero no se implementaron métricas formales de cobertura funcional ni verificaciones específicas.

7 Enlace a repositorio Github

[Repositorio.](#)

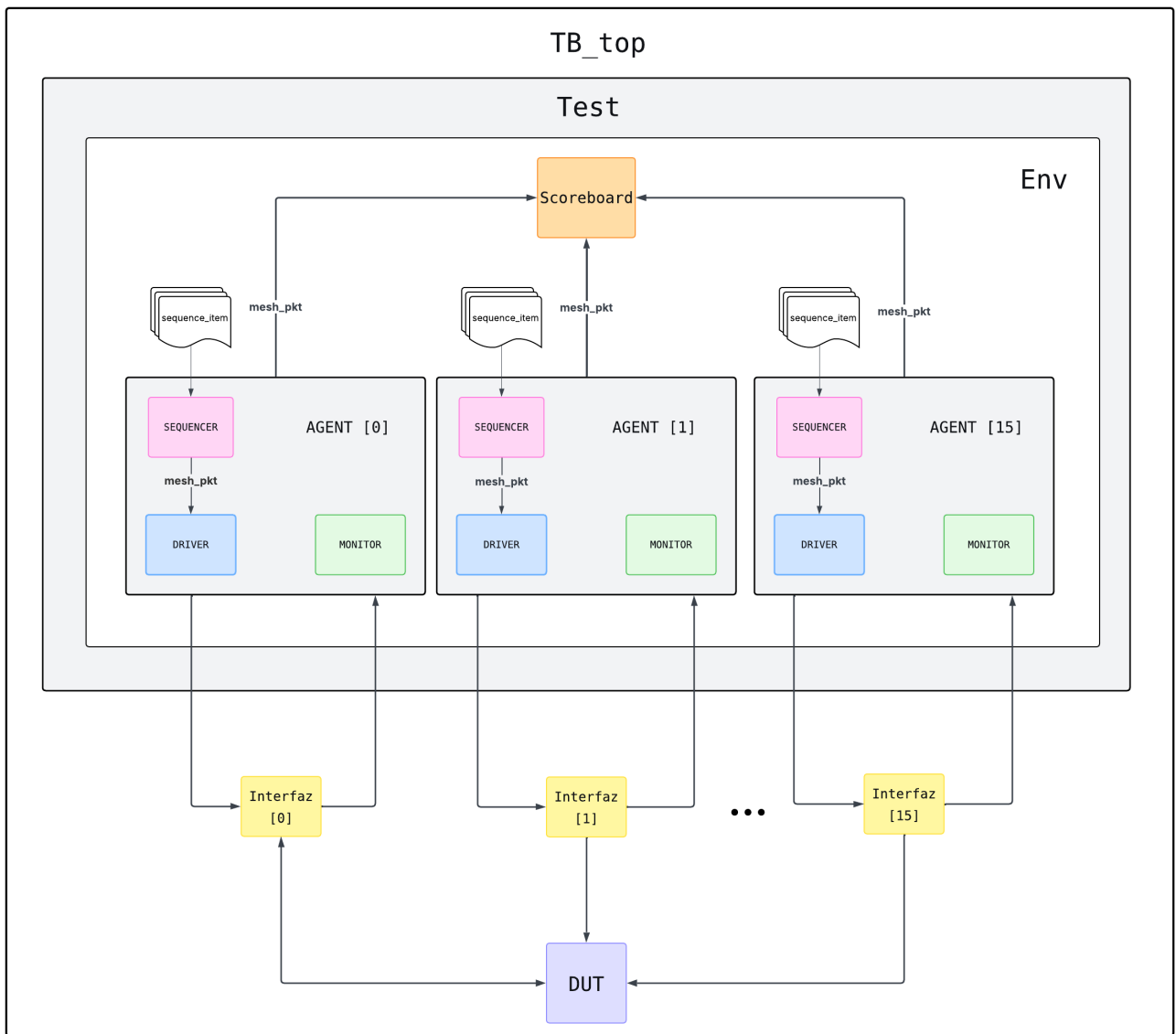


Fig. 3: Diagrama de módulos (transactores), conexiones entre estos y paquetes que se usan para comunicarse entre estos.